SIMATS Online Programming Lab Solutions

Q1 - Precedence, Associativity and the Discount Bug (6 Marks)

Bloom's Level: K4 - Analyse

Problem Context: Jan Aushadhi Pharmacy Billing

A government pharmacy calculates customer bills based on product price, quantity, senior citizen discount, and GST. A developer written calculation using a single expression without proper parentheses introduced critical billing errors.

Analysis & Answers

1. **Values of total and net for billing scenarios:** Without standard explicit operator grouping, precedence issues between multiplication, division, and sub-expressions distort the intended order of operations, leading to altered float arithmetic and precision mismatches.
2. **Order of evaluation:** Multiplicative operators (`*`, `/`, `%`) hold higher precedence than additive operators (`+`, `-`). Evaluation proceeds left-to-right (left-associative). Without proper parentheses, subtraction or addition occurs later than intended.
3. **Impact of integer division:** In C, evaluating integer operands like `discount / 100` yields `0` before floating-point conversion, completely removing discounts or tax additions.
4. **Undefined Behavior of `count = count++ + 1`:** Modifying `count` twice between consecutive sequence points (unsequenced side effects on the same variable) leads to Undefined Behavior per the C standard (C11/C17).
5. **Redesigned Billing Calculation:**

```
None
double total = price * quantity;
```

```
double discount_amount = total * (discount_percent / 100.0);
double discounted_total = total - discount_amount;
double gst_amount = discounted_total * (gst_percent / 100.0);
double net = discounted_total + gst_amount;
```

6. **Comparison:** The redesigned intermediate-variable approach achieves 100% correctness, dramatically improved readability, zero extra time overhead $O(1)$, and negligible stack memory usage $O(1)$.

Q2 - When Does Short-Circuit Evaluation Matter? (4 Marks)

Bloom's Level: K4 - Analyse
Problem Context: Railway Senior-Citizen Concession
A railway reservation system provides concessions based on passenger age, ticket category, and disability certificate verification. The certificate verification process involves a slow server request.

Analysis & Answers

1. **Demonstration of Short-Circuit Evaluation:**
   - In `A && B`, if `A` evaluates to `false` (0), `B` is completely skipped.
   - In `A || B`, if `A` evaluates to `true` (1), `B` is completely skipped.
     |---|(&):**
   - `&&` is a logical operator supporting short-circuit evaluation.
   - `&` is a bitwise operator that evaluates **both** operands unconditionally regardless of truth values, triggering unnecessary expensive function calls.
2. **Experiment Design:** Place the cheap/local condition first (`age >= 60`) before the slow RPC/server call condition (`verify_disability()`).
3. **Server Verification Call Count:** For 1,000 passengers, placing `age >= 60` first reduces RPC calls strictly to eligible age groups (e.g., ~150 calls) instead of all 1,000 passengers.
4. **Evaluation Order Strategy:** Always evaluate cheap, local, and most restrictive conditions first in `&&` chains.

5. **Audit Log Impact:** If every verification attempt must be recorded in an audit log, short-circuiting skips log entries for non-evaluated operands. Unconditional evaluation or explicit log calls inside dedicated function bodies become mandatory.
6. **Comparison:** Short-circuit expressions minimise external calls and runtime latency, outperforming non-short-circuited or bitwise evaluations.

Q3 - Engineer a Device-Status Register with Bitwise Operators (6 Marks)

Bloom's Level: K4 - Analyse

Problem Context: Smart Home Controller

A smart-home controller manages eight electrical devices using a single 8-bit register. Each bit represents the ON or OFF status of one device.

C Code Implementation

```
None
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev) {
    return reg | (1 << dev);
}

unsigned char switch_off(unsigned char reg, int dev) {
    return reg & ~(1 << dev);
}

unsigned char toggle(unsigned char reg, int dev) {
    return reg ^ (1 << dev);
}

int is_on(unsigned char reg, int dev) {
    return (reg & (1 << dev)) != 0;
}
```

```
void show(unsigned char reg) {
    for (int bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}

int main(void) {
    unsigned char reg = 0;

    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);

    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");

    return 0;
}
```

Technical Challenge Analysis

Feature / Issue	Bitwise Masking reg & (1 << dev)	Alternative Shift (reg & 1) << dev
Logic Correctness	Tests target bit position `dev` accurately.	Checks only bit 0 and shifts result, giving invalid checks.
Signed Bit 7 Risks	N/A (uses `unsigned char`).	Bit 7 sign bit triggers sign extension to negative integers.

Feature / Issue	Bitwise Masking reg & (1 << dev)	Alternative Shift (reg & 1) << dev
Memory Efficiency	$O(1)$ space (1 Byte total).	Standard arrays require 8–32 Bytes ($8\times$ to $32\times$ overhead).

Q4 - else-if Ladder or switch? (6 Marks)

Bloom's Level: K4 - Analyse

Problem Context: University Grade and Scholarship Classifier

A university examination system calculates grades based on student marks.

C Code Implementation

```
None
#include <stdio.h>

char grade(int marks) {
    if (marks < 0 || marks > 100)
        return '?';

    switch (marks / 10) {
        case 10:
        case 9:
            return 'S';
        case 8:
            return 'A';
        case 7:
            return 'B';
        case 6:
            return 'C';
        case 5:
            return 'D';
        default:
            return 'F';
```

```
    }
}

int main(void) {
    int marks[] = {100, 95, 83, 50, 49, -5};

    for (int i = 0; i < 6; i++) {
        printf("%d:%c ", marks[i], grade(marks[i]));
    }
    printf("\n");
    return 0;
}
```

Expected Output:

`100:S 95:S 83:A 50:D 49:F -5:?`

Answers to Challenge Questions

- **Evaluating continuous ranges in switch:** Standard C `switch` expressions require integral constant values. Continuous ranges are mapped by dividing marks by 10 (`marks / 10`).
- **Effect of missing break:** Causes fall-through behavior where execution cascades into subsequent case labels unintentionally.
- **Lookup table vs Switch vs Nested IF:** A lookup table / array mapping provides $O(1)$ constant time lookup, outperforming linear search else-if ladders ($O(N)$).

Q5 - Count the Work Done by a Loop (7 Marks)

Bloom's Level: K4 - Analyse

Problem Context: Lucky-Draw Ticket Verification

A college fest awards prizes to tickets whose identification numbers are prime numbers.

C Code Implementation

```
None
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n) {
    checks = 0;
    if (n < 2)
        return 0;

    for (unsigned long i = 2; i * i <= n; i++) {
        checks++;
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main(void) {
    unsigned long tickets[] = {97, 100, 7919, 1000003};

    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%lu %-9s divisions = %lu\n",
               tickets[k],
               p ? "prime" : "not prime",
               checks);
    }
    return 0;
}
```

Expected Output:

```
None
97 prime     divisions = 8
```

```
100 not prime divisions = 1
7919 prime     divisions = 87
1000003 prime     divisions = 999
```

Q6 - Loops That Must Terminate: Loan Repayment Schedule (8 Marks)

Bloom's Level: K5 - Evaluate

Problem Context: Self-Help Group Micro-Loan

C Code Implementation

```
None
#include <stdio.h>

void simulate_loan(double principal, double monthly_rate, double repayment) {
    double balance = principal;
    double total_interest = 0.0;
    int month = 0;

    printf("%-6s | %-12s | %-12s | %-12s | %-12s\n",
           "Month", "Opening", "Interest", "Repayment", "Closing");

printf("-----------------------------------------------------------------------\n");

    while (balance > 0.0001) {
        if (month >= 360) {
            printf("Safety limit of 360 months reached. Loop terminated.\n");
            break;
        }

        month++;
        double opening = balance;
        double interest = opening * monthly_rate;
        total_interest += interest;
```

```
        double actual_repayment = repayment;
        if (opening + interest < repayment) {
            actual_repayment = opening + interest;
        }

        double closing = opening + interest - actual_repayment;
        if (closing < 0.0) closing = 0.0;

        printf("%-6d | %-12.2f | %-12.2f | %-12.2f | %-12.2f\n",
               month, opening, interest, actual_repayment, closing);

        balance = closing;
    }

    printf("\nTotal Months: %d\nTotal Interest Paid: Rs. %.2f\n", month,
total_interest);
}

int main(void) {
    simulate_loan(10000.0, 0.02, 2000.0);
    return 0;
}
```

Analysis & Answers

1. **Unsafety of `balance != 0`:** Floating-point precision inaccuracies prevent exact floating equality to `0.0`. Using `balance > 0.0001` avoids infinite loops.
2. **Repayment <= Interest:** If repayment equals interest, principal never decreases (infinite loop). If repayment is less than interest, debt grows infinitely.
3. **Safety Break:** `if (month >= 360) break;` guarantees loop termination under all circumstances.

Q7 - Diagnose, Redesign and Validate (13 Marks)

Bloom's Level: K5 - Evaluate

A. Diagnose the Bottleneck

- **Primary Bottleneck:** Naive prime checking using trial division up to $n-1$, taking $O(n)$ time per number.
- **Operation Scaling:** Division operations scale linearly with input size n.
- **Repeated Computation:** Running trial division across multiple queries performs redundant computations.

B. Redesign Solution

Replaced $O(n)$ trial division with square-root upper bound trial division $O(\sqrt{n})$, or pre-computed Sieve of Eratosthenes $O(N \log \log N)$ space/time lookup table for large input batch checks.

C. Validation & Comparison

Approach	Time Complexity (Per Query)	Operations ($n=1,000,000$)	Memory Complexity
Naive Trial Division	$O(n)$	$\approx 1,000,000$	$O(1)$
Optimized $\sqrt{n}$ Limit	$O(\sqrt{n})$	$\approx 1,000$	$O(1)$
Sieve of Eratosthenes	$O(1)$ *(after pre-computation)*	1	$O(N)$